Alpha Evolve Clone - CSCE 585

John Aydin, Anthony Goldhammer, Tiffany Yu
2025

Dec



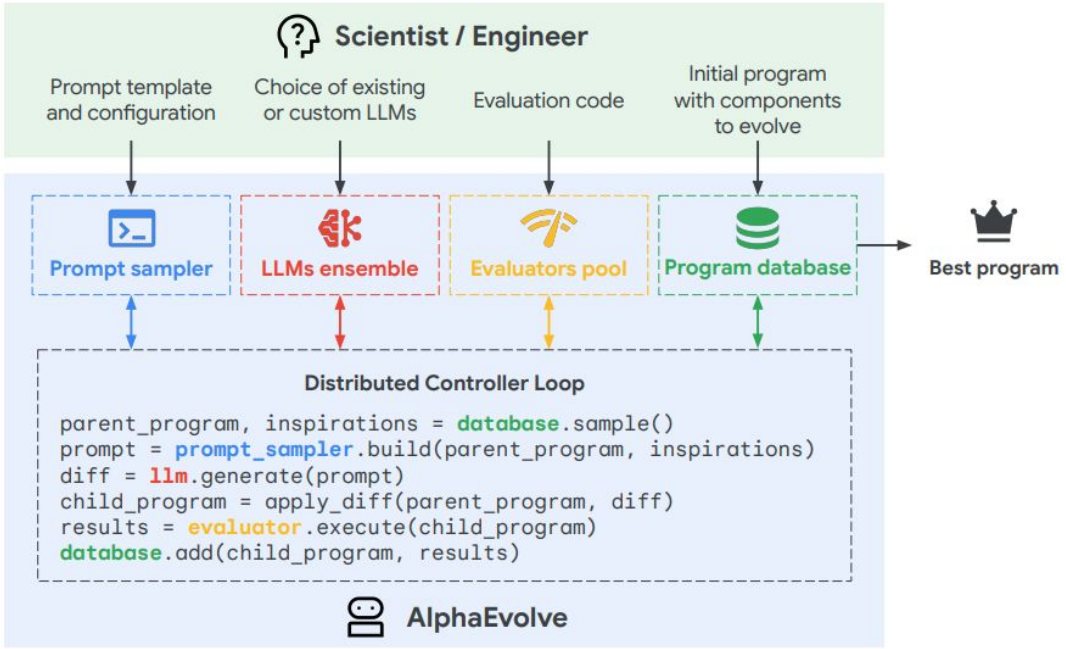
UNIVERSITY OF
South Carolina

Table of Contents

1. Motivation
2. Our Approach
3. Results
4. Demo
5. Results on Our Data

Inspiration from AlphaEvolve

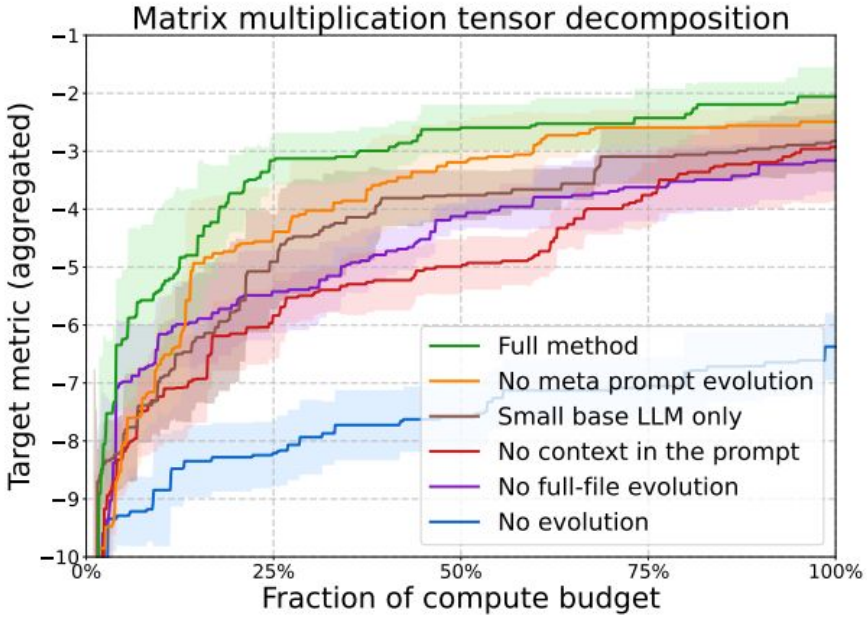
Our project, an LLM-driven evolutionary algorithm to optimize code, borrows its architecture from AlphaEvolve [1].



Overview of AlphaEvolve's [1] architecture.

Why this Project?

The AlphaEvolve paper **does not** explicitly state their compute budgets, making us question how this approach would perform at a small scale.

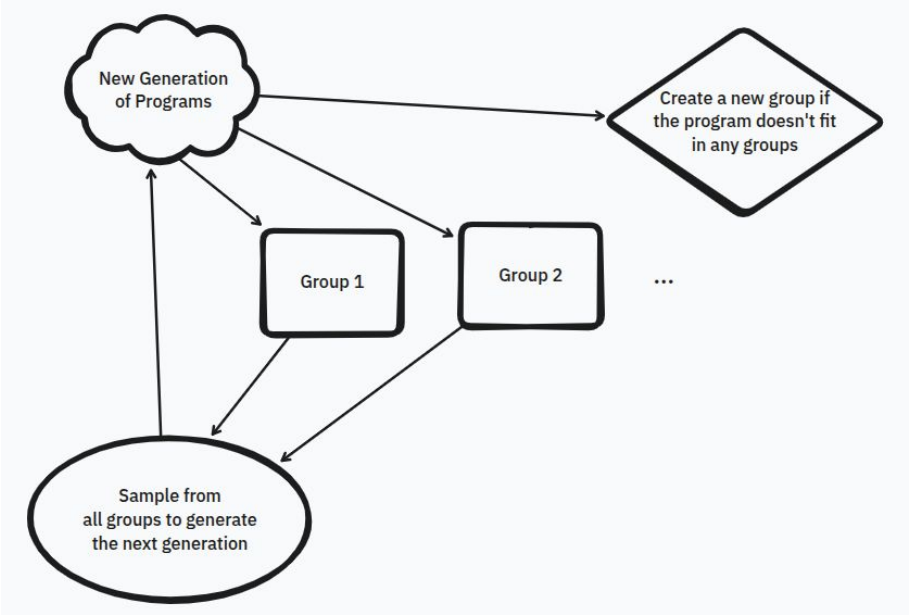


AlphaEvolve's [1] ablation study. Notice the X axis is given as a fraction of the total budget.

Our Approach – Program Database

We implement a program database that aims to maximize the diversity of the candidate programs chosen in each generation.

We compute the similarity of the programs by comparing their abstract syntax trees (AST) via pq-gram distance [2]



Our Approach – Models

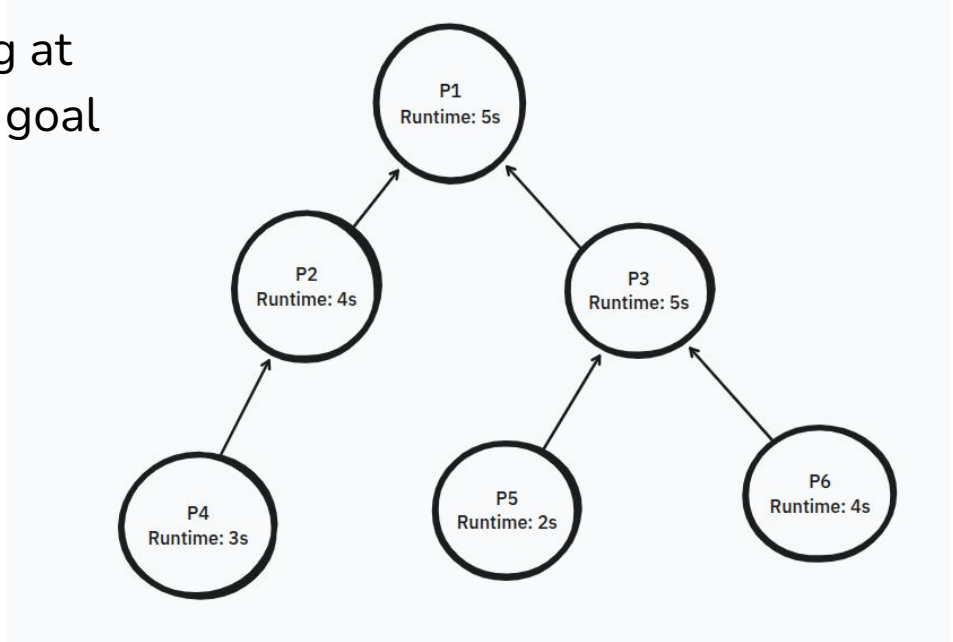
The final version of the our project uses Qwen2.5-Coder-1.5B-Instruct [3], but we tested several other models including:

- VibeThinker-1.5B [5]: Qwen2.5-Coder fine-tuned for CoT reasoning
- Gemma-3-270m-it [4]: General model designed for edge devices

All models were run with vLLM on a 4060ti 16GB, a low end consumer graphics card.

Our Approach – Prompt Sampler

Each prompt is created by looking at a program and its ancestors. The goal of this is that the model will be informed on what changes were made to improve the program between generations.

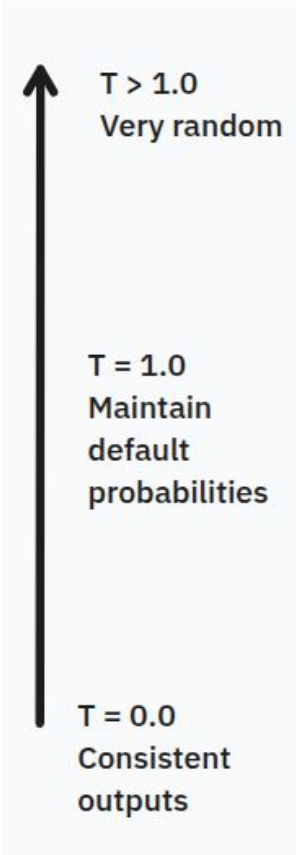


Our Approach – Controller

The controller's goal is to maintain progress in the search for an optimal program. One way we aim to achieve this is by giving it control of the temperature (T) of the program database and LLM(s).

The final implementation reacts to the number of passing programs in each generation to increase or decrease the temperature

$$P_i = \frac{e^{\frac{y_i}{T}}}{\sum_{k=1}^{n} e^{\frac{y_k}{T}}}$$



Definition of softmax and the effect T (temperature) has on probabilities

Results – 3sum

3sum doesn't have a better algorithm than the baseline, so our system could beat Gemini and GPT-5.1.

	Runtime (to find the algorithm)	Leetcode Percentile	Runtime (Leetcode)
Gemini 2.5 Pro	31.170s	44.69%	595 ms
GPT-5.1	**6.324s**	71.15%	512ms
Our System	142.249s	**89.91%**	**424ms**

Performance of baselines (5th percentile implementation from LeetCode) compared to our system, 10 generations, 16 candidates per generation, one ancestor shown in prompt, target fail rate of 0.5

Results – BitWiseOR

Gemini and GPT-5.1 found the standard set-based DP algorithm that tracks ORs of all subarrays ending at each index. This avoids the baseline $O(n^2)$ behavior and runs in $O(n \cdot k)$ with very small k. Our system did not find this improved algorithm.

	Runtime (to find the algorithm)	Leetcode Percentile	Runtime (Leetcode)
Gemini 2.5 Pro	37.192s	77.78%	426ms
GPT-5.1	**5.504s**	**92.15%**	**411 ms**
Our System	81.194s	56.33%	480 ms

Performance of baselines (5th percentile implementation from LeetCode) compared to our system, 10 generations, 16 candidates per generation, one ancestor shown in prompt, target fail rate of 0.5

Results – 24 Game

24 Game has a stronger DFS-based solution. That has very little overlap with the baseline we provided in the prompt. Gemini and GPT-5.1 found it; our system did not, we improved the baseline but not the underlying algorithm.

	Runtime (to find the algorithm)	Leetcode Percentile	Runtime (Leetcode)
Gemini 2.5 Pro	52.687s	71.74%	32ms
GPT-5.1	**33.129**	**87.43%**	**26ms**
Our System	197.801s	53.25%	48ms

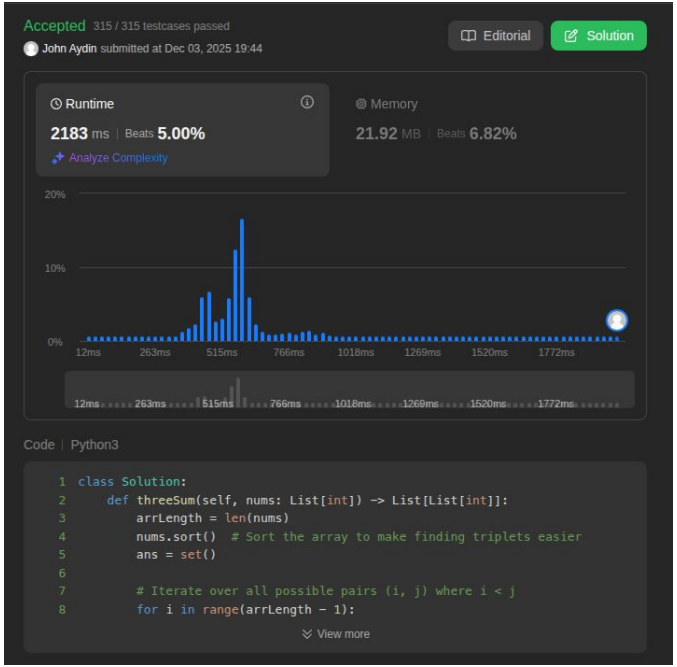
Performance of baselines (5th percentile implementation from LeetCode) compared to our system, 10 generations, 16 candidates per generation, one ancestor shown in prompt, target fail rate of 0.5

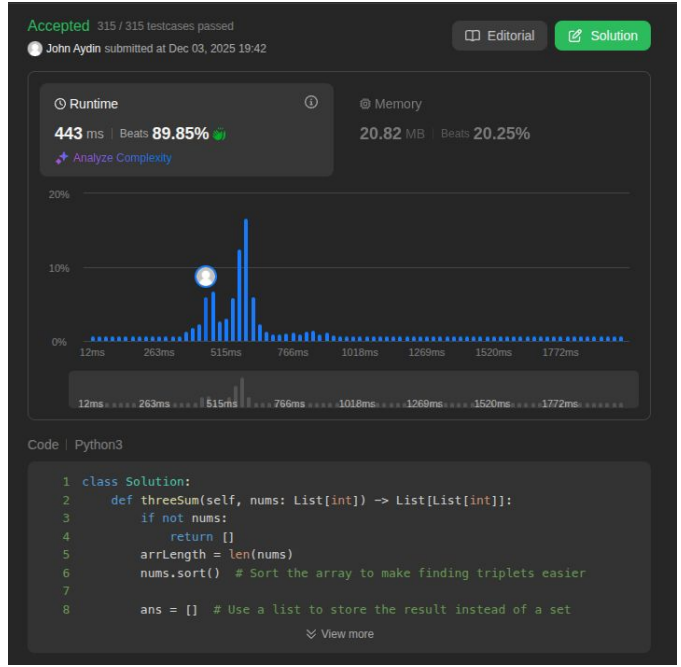
Results - 3sum / Demo

The key things to pay attention to in the output are the best overall score, number of bins, and the number of ancestors of the final program.



Results - 3sum





Discussion and Limitations

Our implementation outperformed the baselines when the provided code that already contained the optimal algorithm, but failed when there was a better algorithm than the baseline.

This indicates that our system is good at optimizing code through small improvements built up over several iterations, but does not make large breakthroughs through full rewrites of the program.

We think this can be improved by using an ensemble of models, prompting larger or CoT models for full rewrites of the code, and letting the smaller models make smaller improvements.

Discussion and Limitations

The current implementation does not hold up past ~50 generations due to an interaction between the program database and temperature controller.

Code + References

References:
[1] A. Novikov et al., “AlphaEvolve: A coding agent for scientific and algorithmic discovery,” arXiv preprint arXiv:2506.13131, 2025.
[2] N. Augsten, M. Böhlen, and J. Gamper, “The pq-gram distance between ordered labeled trees,” ACM Transactions on Database Systems, vol. 35, no. 1, Article 4, pp. 1–36, 2010, doi: 10.1145/1670243.1670247
[3] Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., … Lin, J. (2024). Qwen2.5-Coder Technical Report. arXiv. https://doi.org/10.48550/arXiv.2409.12186
[4] Gemma Team. (2025). Gemma 3 Technical Report. arXiv. https://doi.org/10.48550/arXiv.2503.19786
[5] Xu, S., Zhou, Y., Wang, W., Min, J., Yin, Z., Dai, Y., Liu, S., Pang, L., Chen, Y., & Zhang, J. (2025). Tiny Model, Big Logic: Diversity-Driven Optimization Elicits Large-Model Reasoning Ability in VibeThinker-1.5B. arXiv. https://doi.org/10.48550/arXiv.2511.06221

Code:
https://github.com/csce585-mlsystems/alpha-evolve-clone

01.

LLMs as Evolutionary Optimizers

02.

Inspiration from Alpha Evolve

03.

Our Method

What are we trying to achieve?

The main goal with this system is to benefit from a large number of calls to the LLM. This is achieved through an evolutionary algorithm in which the LLM proposes mutations to candidate programs based on fitness feedback.

LLM Ensemble

Since we shifted this project to run locally, the LLM ensemble is just a single LLM: Qwen/Qwen2.5-Coder-1.5B, although we also test a variant fine-tuned for reasoning: WeiboAI/VibeThinker-1.5B

This model is run locally on a 16GB 4060 ti with vLLM.

Program Database

The most significant contribution we have is in the program database. While AlphaEvolve uses multiple metrics to achieve diversity in solutions, we employ a custom program database that uses the abstract syntax tree (AST) editing distance group like programs. Each group

02.

Use Cases

04.

Limitations

Magic Numbers and Configurability

One major limitation of our current approach is the amount of “magic numbers” / points of configuration.

Parameter Name	Component	Description	Problem
Similarity Threshold	Program Database	Controls when bins are created	Highly dependent on input program
Default temp	Controller	The starting temperature of the model	Dependent on the model used
Temp step size	Controller	The amount that the model temperature is changed to reach the target fail rate	There is likely a clever algorithm to solve this problem automatically.
Target fail rate	Controller	The rate at which we want the model to fail to generate a passing solution	This number is completely arbitrary and is only to keep the temperature under control